

4. 图与网络分析

4.1 实验目的

- (1) 掌握最短路问题、最大流问题和最小生成树问题的基本模型；
- (2) 熟练使用 OPL 语言求解最短路问题、最大流问题和最小生成树问题。
- (3) 学会建立最短路问题、最大流问题和最小生成树问题的模型。

4.2 最短路问题

4.2.1 数学模型的建立

已知一个网络图 $G(v, e, w)$ 如图 4-1 所示(v 是顶点集合, e 是边的集合, w 是网络图的权矩阵), 找出某一对顶点之间的最短路径。也就是说在两个点之间的所有路径中, 找到路径上的边的权值之和最小的路径。网络图可以是有向的, 也可以是无向的。通常我们把无向网络图看成是每两个顶点之间有双向边的有向网络图。

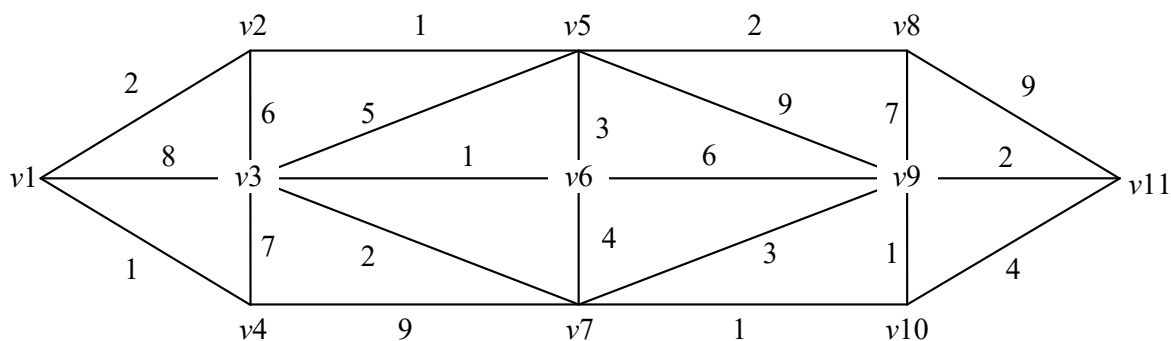


图 4-1 网络图

比如, 从 $v1$ 到 $v11$ 存在很多路径, $(v1, v2, v5, v8, v11)$ 是一条路径, $(v1, v3, v5, v8, v11)$ 也是一条路径, 还有很多这样的路径。最短路径是把各个路径上边的权值相加, 其中和最小的路径。

设 x_{ij} 为对应于网络图的边 $\langle i, j \rangle$ 在不在最短路径上的决策: 当顶点之间的路径上含边 $\langle i, j \rangle$ 时, $x_{ij}=1$; 否则 $x_{ij}=0$ 。

在 $x_{ij}=1$ ($\langle i, j \rangle$ 属于 E) 的所有组合中, 找到一个组合, 使 $x_{ij}=1$ 的 $\langle i, j \rangle$ 边的权值之和最小, 用表达式表示则为 $\sum w_{ij}x_{ij}$ 最小的组合。如果没有任何约束, 那么所有的边 $\langle i, j \rangle$ 属于 E , 对应的 $x_{ij}=1$, $\sum w_{ij}x_{ij}$ 权值之和最小为 0, 因为没有边的 $x_{ij}=1$, 也就意味着没有边在最短路径中。因此, 需要添加约束条件。

需要有若干条边的 x_{ij} 等于 1。 $x_{ij}=1$ 的所有边 $\langle i, j \rangle$ 应该构成一条从始点到终点的路

径。这条路径不一定包含所有的结点，但至少包含始点和终点。不论结点在不在最短路径上，对于每个结点，都有如下关系成立。

(1) 对于始点 1: ① $x_{1k}=1$ (k 是其中一个结点)，以 1 为起点的弧中，只能有一条弧出现在最短路径上。② $x_{j1}=1$ ($j=1,2,\dots,n$)；以 1 为终点的弧中，不能有弧出现在最短路径上，否则就构成了回路。

(2) 对于终点 n : ① $x_{kn}=1$ (k 是其中一个结点)，以 n 为终点的弧中，只能有一条弧出现在最短路径上。② $x_{nj}=1$ ($j=1,2,\dots,n$)；以 n 为起点的弧中，不能有弧出现在最短路径上，否则就构成了回路。

(3) 除了始点和终点之外的中间点 i 所在的所有边中，或者所有边都不出现在最短路径上，即所有边所对应的 x_{ij} 和 $x_{ji}=0$ ($j=1,2,\dots,n$)；或者有以 i 为起点和终点的边，各有一个出现在最短路径上，即 $x_{ik}=1$ 和 $x_{ji}=1$ (k 和 j 是其中的两个结点)。

通过总结，在有向图的最短路径上的顶点和弧有如下特点：

(1) 始点出现在起点的次数与始点出现在终点的次数差为 1。 -

(2) 终点出现在终点的次数与终点出现在起点的次数差为 1。

(3) 其他任意点出现在起点和终点的次数相同(包括从未出现在起点和终点的情况)。这些看似松散的约束条件与极小化目标函数(最短路径上弧的权值之和最小)配合，能够实现找到最短路径的目标。

其数学模型为

$$\begin{aligned} \min & \sum_{\langle i,j \rangle \in E} w_{ij} x_{ij} \\ \text{s.t.} & \begin{cases} \sum_{j \in V, \langle i,j \rangle \in E} x_{ij} - \sum_{j \in V, \langle j,i \rangle \in E} x_{ji} = \begin{cases} 1, (i=1) \\ -1, (i=n) \\ 0, (i \neq 1, n) \end{cases} \\ x_{ij} = \{0,1\}, i \in V, j \in V, \langle i,j \rangle \in E \end{cases} \end{aligned}$$

4.2.2 最短路径问题的 OPL 模型

(1) 数据的定义和初始化。定义变量 n 存储顶点个数。定义区间变量 v 表示邻接矩阵下标。定义二维数组 $\text{adj}[v][v]$ 存储邻接矩阵。定义二维数组 $w[v][v]$ 存储权值矩阵。

`int n=...`;

`range v=1..n;`

`int adj[v][v]=...`;

```
int w[v][v]=...;
```

(2) 决策变量的定义。定义布尔型二维数组 $x[v][v]$ 的值表示顶点 i 和顶点 j 在最短路径上。如果顶点 i 和 j 在最短路径上，则 $x[i][j]=1$ ；否则 $x[i][j]=0$ 。

```
dvar boolean x[v][v];
```

(3) 目标函数的定义。在最短路径上的边的权值之和最小。

```
minimize sum(i in v,j in v)x[i][j]*w[i][j];
```

(4) 约束条件的定义。

约束条件 $\sum_{j \in V, \langle i, j \rangle \in E} x_{ij} - \sum_{j \in V, \langle i, j \rangle \in E} x_{ji} = 1, i=1$ 所对应的 OPL 表达式为：

```
sum(j in v)adj[j][1]*x[j][1]+1==sum(k in v)adj[1][k]*x[1][k];
```

约束条件 $\sum_{j \in V, \langle i, j \rangle \in E} x_{ij} - \sum_{j \in V, \langle i, j \rangle \in E} x_{ji} = -1, i=n$ 所对应的 OPL 表达式为：

```
sum(j in v)adj[j][11]*x[j][11]-1==sum(k in v)adj[11][k]*x[11][k];
```

约束条件 $\sum_{j \in V, \langle i, j \rangle \in E} x_{ij} - \sum_{j \in V, \langle i, j \rangle \in E} x_{ji} = 0, i \neq 1, n$ 表示除始点和终点外，任何一个中间点的

所有的边中，选中的以该点为起点的边数和等于以该点为终点的边数和，对应的 OPL 表达式为：

```
forall(i in v:i!=1&&i!=n)sum(j in v)adj[j][i]*x[j][i]==sum(k in v)adj[i][k]*x[i][k];
```

合并约束条件如下：

```
subject to{
```

```
sum(j in v)adj[j][1]*x[j][1]+1==sum(k in v)adj[1][k]*x[1][k];
```

```
sum(j in v)adj[j][11]*x[j][11]-1==sum(k in v)adj[11][k]*x[11][k];
```

```
forall(i in v:i!=1&&i!=n)sum(j in v)adj[j][i]*x[j][i]==sum(k in v)adj[i][k]*x[i][k];
```

```
}
```

4.2.3 案例 1

求图 4-1 的无向图中从 v_1 到 v_{11} 之间的最短路径。

定义数据文件内容如下

```
adj=[[0 1 1 1 0 0 0 0 0 0 0]
```

```
[1 0 1 0 1 0 0 0 0 0 0]
```

```
[1 1 0 1 1 1 1 0 0 0 0]
```

```
[1 0 1 0 0 0 1 0 0 0 0]
[0 1 1 0 0 1 0 1 1 0 0]
[0 0 1 0 1 0 1 0 1 0 0]
[0 0 1 1 0 1 0 0 1 1 0]
[0 0 0 0 1 0 0 0 1 0 1]
[0 0 0 0 1 1 1 1 0 1 1]
[0 0 0 0 0 0 1 0 1 0 1]
[0 0 0 0 0 0 0 1 1 1 0]];
w=[[0 2 8 1 0 0 0 0 0 0 0]
[2 0 6 0 1 0 0 0 0 0 0]
[8 6 0 7 5 1 2 0 0 0 0]
[1 0 7 0 0 0 9 0 0 0 0]
[0 1 5 0 0 3 0 2 9 0 0]
[0 0 1 0 3 0 4 0 6 0 0]
[0 0 2 9 0 4 0 0 3 1 0]
[0 0 0 0 2 0 0 0 7 0 9]
[0 0 0 0 9 6 3 7 0 1 2]
[0 0 0 0 0 0 1 0 1 0 4]
[0 0 0 0 0 0 0 9 2 4 0]];
```

运行最短路问题的 OPL 模型，得到求解结果如下，最短距离为 13.

```
x = [[0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 1]
[0 0 0 0 0 0 0 0 1 0 0]
```

[0 0 0 0 0 0 0 0 0 0];

最短路径为 $v_1 \rightarrow v_2 \rightarrow v_5 \rightarrow v_6 \rightarrow v_3 \rightarrow v_7 \rightarrow v_{10} \rightarrow v_9 \rightarrow v_{11}$ ，见图 4-2.

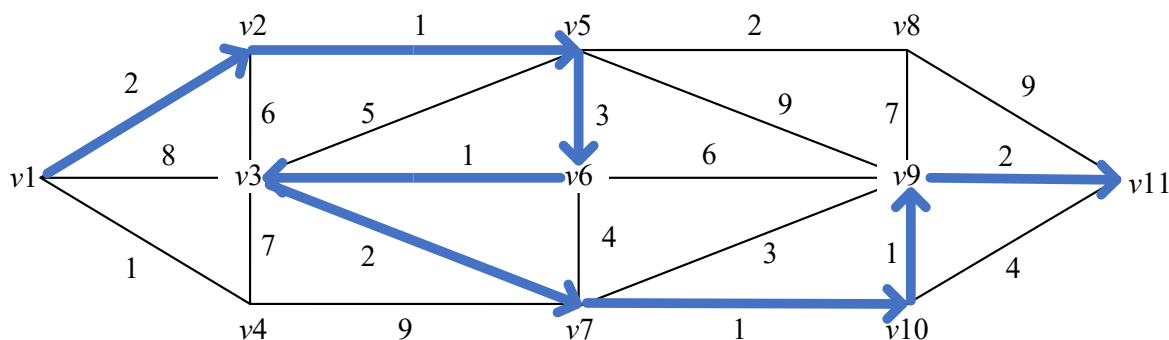


图 4-2 最短路径

4.3 最大流问题

最大流问题是一类应用极为广泛的问题，例如在交通运输网络中有人流、车流、货物流，供水网络中有水流，金融系统中有现金流，通信系统中有信息流，等等。20 世纪 50 年代，福特（Ford）、富克逊（Fulkerson）建立的“网络流理论”是网络应用的重要组成部分。

4.3.1 数学模型的建立

设一个赋权有向图 $D=(V,E)$ ，在 V 中指定一个发点 v_s 和一个收点 v_t ，其他的点叫做中间点。对于 D 中的每一个弧 $(v_i, v_j) \in E$ ，都有一个非负数 c_{ij} ，叫做弧的容量。我们把这样的图 D 叫做一个容量网络，简称网络。记作 $D=(V,E,C)$ ，如图 4-3 所示。

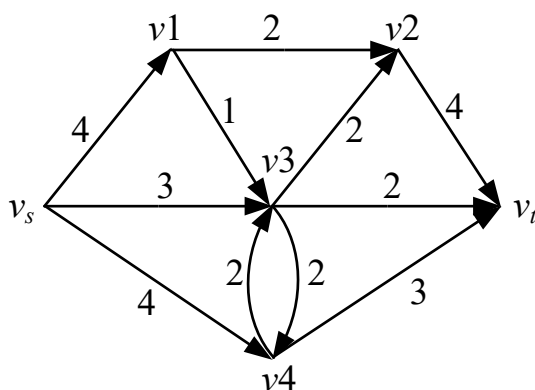


图 4-3 网络图

网络 D 上的流是指定义在弧集合 E 上的一个函数，其中 $f(v_i, v_j) = f_{ij}$ ，叫做弧 (v_i, v_j) 上的流量。满足下列条件的流称为可行流

(1) 容量条件：对于每一个弧 $(v_i, v_j) \in E$ ，有 $0 \leq f_{ij} \leq c_{ij}$ 。

(2) 平衡条件：对于发点 v_s ，有 $\sum_{(v_s, v_j) \in E} f_{sj} - \sum_{(v_j, v_s) \in E} f_{js} = -v$

对于收点 v_t ，有 $\sum_{(v_j, v_t) \in E} f_{jt} - \sum_{(v_t, v_j) \in E} f_{jt} = v$

对于中间点，有 $\sum_{(v_i, v_j) \in E} f_{ij} - \sum_{(v_i, v_j) \in E} f_{ji} = 0$

可行流中 $f_{ij} = c_{ij}$ 的弧叫做饱和弧， $f_{ij} < c_{ij}$ 的弧叫做非饱和弧。 $f_{ij} > 0$ 的弧叫做非零流弧， $f_{ij} = 0$ 的弧叫做零流弧。

因此，最大流问题的数学模型为

$$\begin{aligned} \max v \\ \sum_{(i,j) \in E} x_{ij} - \sum_{(i,j) \in E} x_{ji} &= \begin{cases} v, i = s \\ -v, i = t \\ 0, i \neq s, t \end{cases} \\ 0 \leq x_{ij} \leq c_{ij}, \forall (i, j) \in E \end{aligned}$$

4.3.2 最大流问题的 OPL 模型

(1) 数据的定义和初始化。首先要确定初始化网络图的节点数和弧的容量。定义字符串集合 `nodes`，存储顶点名称。定义结构体 `edge`，表示有向弧。定义 `edge` 类型的集合 `edges`，存储网络图中的有向弧。定义数组 `cap[edges]`，表示各个有向弧的容量。

```
{string} nodes=...;
tuple edge{
string origin;
string destination;
}
setof(edge) edges=...;
int cap[edges]=...;
```

(2) 决策变量的定义。

定义整型一维数组 `flow[edges]`，表示各个有向弧的流量。

```
dvar int+ flow[edges];
```

定义决策表达式 `maxflow`，表示始点流出的流量之和。

```
dexpr int maxflow=sum(i in edges:i.origin=="begin") flow[i];
```

(3) 目标函数的定义。定义目标函数是整个网络中的流量最大。

```
maximize maxflow;
```

(4) 约束条件的定义

对于中间点来说，流出的流量和流入的流量平衡，约束如下

```
forall(i in nodes:i!="begin"&&i!="end")
```

```
    sum(j in edges:j.origin==i)flow[j]==sum(j in edges:j.destination==i)flow[j];
```

每条弧上的流量不超过其容量，约束如下

```
forall(i in edges)flow[i]<=cap[i];
```

合并约束条件如下：

```
subject to{
```

```
forall(i in nodes:i!="begin"&&i!="end")
```

```
    sum(j in edges:j.origin==i)flow[j]==sum(j in edges:j.destination==i)flow[j];
```

```
forall(i in edges)flow[i]<=cap[i];
```

4.3.3 案例 2

求图 4-3 的网络图中的最大流。

定义数据文件内容如下

```
nodes={begin,one,two,three,four,end};
```

```
edges={<begin,one>,<begin,three>,<begin,four>,<one,two>,<one,three>,<two,end>,<three,two>,<three,four>,<three,end>,<four,three>,<four,end>};
```

```
cap=[4,3,4,2,1,4,2,2,3,2,3];
```

运行最大流问题的 OPL 模型，得到结果如下，最大流量为 9，见过见图 4-4.

```
flow = [3 3 3 2 1 4 2 0 2 0 3];
```

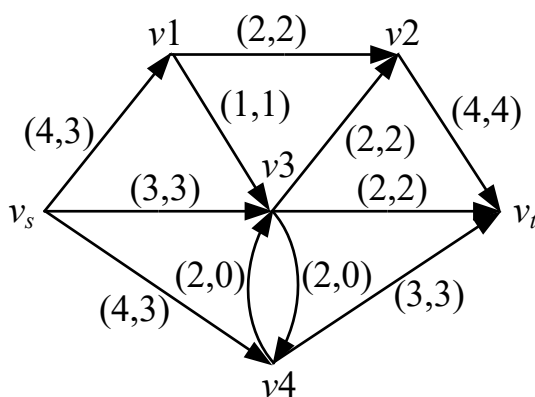


图 4-4 最大流

4.4 练习

在图 4-5 所示道路网中，要求沿道路铺设光纤网使点 v1 与其他各点可以进行通讯，求如何使得光纤的总长最小。

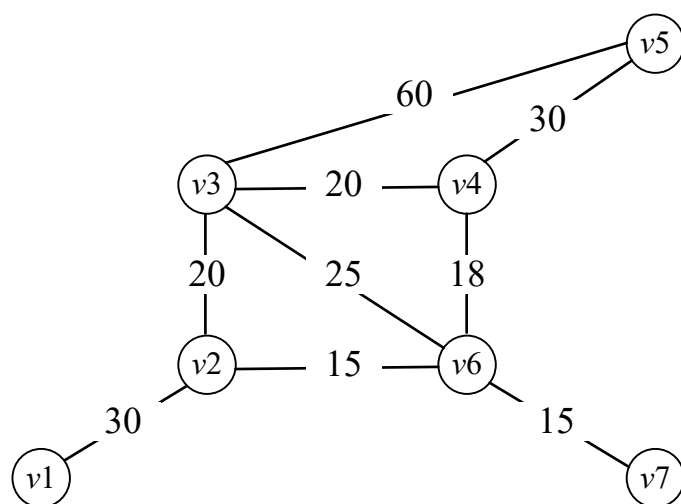


图 4-5 网络图